



Sardar Vallabhbhai National Institute of Technology (SVNIT),

Surat Department of Artificial Intelligence

Intelligent Multi-Agent and Expert Systems (AI401), Semester VII

U23AI052 Submission File

Instructions

1. Save your file as LAB_No_Roll_No.doc (e.g. LAB_1_U21AI001.doc).
2. Paste a snapshot of each step followed along with the result(s).
3. Record your observations and comments after the results.

Experiment 1

Knowledge Representation — Semantic Network with Inheritance

Aim

To represent knowledge as a semantic network of (subject, relation, object) triples and answer queries using property inheritance along is-a links, including exceptions.

Theory

Before an expert system can reason, knowledge must be encoded in a machine-usable form. A semantic network represents concepts as nodes and relationships as labelled, directed edges. The is-a relation organises concepts into a taxonomy, and property inheritance lets specific concepts acquire the properties of their ancestors: a shark has skin because a shark is-a fish is-a animal, and animals have skin.

Inheritance must allow exceptions: the default 'birds can fly' is overridden for penguins by a more specific 'cannot fly' fact. The query procedure therefore searches from the most specific node upward and lets local knowledge win — the same default-with-exceptions behaviour found in frame systems, where each concept is a frame of slots and fillers. This experiment establishes the knowledge-representation foundation that every later experiment builds on: the production rules of Experiment 2 and the chaining engines of Experiments 3 and 4 all reason over knowledge encoded in exactly this declarative style.

Algorithm

1. Store the network as a list of (subject, relation, object) triples.

```
Semantic network triples:
('animal', 'has', 'skin')
('animal', 'can', 'breathe')
('bird', 'is-a', 'animal')
('bird', 'has', 'feathers')
('bird', 'can', 'fly')
('penguin', 'is-a', 'bird')
('penguin', 'cannot', 'fly')
```

2. To find the inheritance chain of a node, repeatedly follow its is-a link until no parent exists.
3. To answer query(node, relation), walk up the chain; at each level check for an exception ('cannot') and then for a local value; return the first value found.
4. Report 'unknown' if the chain is exhausted.

Procedure and Result

Paste the snapshots of the steps followed and the output produced by the semantic network program below, including the knowledge base of triples, the answer returned for each query, and the inheritance chain obtained for at least one node.

Observations

Comment on how the query for 'penguin can' returns 'cannot fly' even though birds can fly, on why exceptions must be checked before inherited values at every level of the chain, and on what the result tells you about default reasoning being non-monotonic.

Observations and results i got: The semantic network successfully demonstrates inheritance through is-a links. The query for penguin can returns cannot fly because local exceptions are checked before inherited properties. This illustrates non-monotonic reasoning, where more specific knowledge overrides general knowledge. The frame implementation produces equivalent inheritance behavior using ako links.

Exercises

1. Extend the network with an ostrich (a bird that cannot fly but can run fast) and verify the queries.

```
Exercise 1:
ostrich inheritance chain:
ostrich -> bird -> animal

Semantic network queries:
ostrich can          = cannot fly
ostrich has          = feathers
ostrich runs         = fast

Frame queries:
ostrich can          = cannot fly
ostrich has          = feathers
ostrich runs         = fast
```

2. Re-implement the same knowledge as frames: a Python dict per concept with slots, fillers and an 'ako' (a-kind-of) slot, and write the inheritance lookup for it.
3. Allow multiple parents (multiple is-a links) and decide, with justification, the order in which they are searched.

Exercise 3:

ostrich direct parents:

bird, runner

search order: depth-first, left-to-right

Semantic network queries:

ostrich can = cannot fly

ostrich has = feathers

ostrich runs = fast

Frame queries:

ostrich can = cannot fly

ostrich has = feathers

ostrich runs = fast

